

Regression Tree

Report tecnico del progetto di Metodi Avanzati di Programmazione, anno accademico 2025/2026.

Gruppo

- Davide Fornelli
- Nicola Cialdella
- Francesco Cavaliere

Introduzione al Regression Tree

Un albero di regressione è un modello supervisionato che stima un valore numerico. Il training set contiene attributi esplicativi e un target continuo.

Ogni nodo interno applica un test a un attributo. I rami rappresentano gli esiti del test; ogni foglia restituisce la media dei target degli esempi raggiunti.

La costruzione avviene per partizionamento ricorsivo. A ogni passo viene scelto lo split che riduce maggiormente l'errore quadratico.

Introduzione al progetto

Il progetto apprende alberi di regressione da file `.dat` o da tabelle MySQL/MariaDB.

Comprende tre applicazioni:

- server MAP con accesso JDBC;
- client testuale per training, caricamento e predizione;
- GUI Swing per training e visualizzazione dell'albero.

Componenti

Server

`mapServer.jar` legge una tabella del database, costruisce l'albero, salva l'archivio e gestisce le predizioni.

Il server usa socket TCP e stream di oggetti Java. Ogni connessione è servita da un thread `ServerOneClient`.

Client da console

`mapClient.jar` si collega al server. L'utente può:

- apprendere un albero da una tabella;
- caricare un archivio esistente;
- scegliere i rami fino alla predizione.

Client con GUI

`mapGUI.jar` è realizzato con Swing e AWT. Offre due origini:

- **File locale:** legge un file `.dat` senza server;
- **Database:** chiede al server di apprendere una tabella.

La GUI disegna nodi, rami e predizioni. Include zoom, centratura e dettagli testuali. Non esegue la predizione guidata.

Flussi

```
file .dat → mapGUI.jar → training locale → albero grafico

MySQL ⇌ mapServer.jar ⇌ mapGUI.jar → albero grafico
                      ⇌ mapClient.jar → predizione
```

Apprendimento

Per una partizione S , la foglia usa la media dei target:

$$\hat{y}_S = (1 / |S|) \cdot \sum y_i$$

Il costo è la somma degli errori quadratici:

$$SSE(S) = \sum (y_i - \hat{y}_S)^2$$

Il codice confronta gli split validi e sceglie quello con la somma delle SSE dei figli più bassa.

1. caricamento e validazione dei dati;
2. ordinamento degli esempi rispetto all'attributo;
3. calcolo dei possibili split;
4. creazione ricorsiva dei sotto-alberi;
5. creazione della foglia e calcolo della media.

Dati da file

Il formato locale dichiara schema, attributi, target e righe.

```
@schema 2
@desc X A,B
@desc Y
@target class
@data 4
A,2.0,1.0
A,5.0,1.5
B,6.0,10.0
B,10.0,10.0
```

Dati da database

Lo schema SQL determina attributi discreti e continui. L'ultima colonna deve essere numerica e viene usata come target.

Struttura del codice

Modello

- **data**: lettura, attributi e validazione;
- **tree**: nodi, split, foglie e albero;
- **database**: connessione JDBC, schema e righe;
- **server**: protocollo e sessioni concorrenti;
- **estensioni.gui**: finestra, canvas e modello visuale.

Classi principali

- **Data** conserva esempi e metadati;
- **RegressionTree** costruisce, salva e carica il modello;
- **SplitNode** rappresenta un test;
- **LeafNode** conserva la predizione;
- **RegressionTreeGUI** coordina input e visualizzazione;
- **RegressionTreeCanvas** calcola il layout e disegna l'albero.

Visualizzazione remota

Il protocollo server non esporta direttamente l'albero. La GUI percorre i cammini di predizione e ricostruisce un **VisualTree**. Il limite è 10.000 nodi e il timeout di rete è 30 secondi.

Persistenza

Il training da database serializza l'albero in un file con il nome della tabella. Il client testuale può ricaricarlo in una sessione successiva.

I diagrammi completi sono in `docs/diagramma-classi.pdf` e `docs/diagramma-classi-client-server.pdf`.

Testing

I test automatici usano JUnit 4 e verificano il modello dati, l'apprendimento, gli errori di input, la tastiera e la GUI.

Aree verificate

- lettura e validazione dei file `.dat`;
- attributi discreti e continui;
- costruzione di nodi e foglie;
- split e predizioni;
- layout, zoom e statistiche della GUI;
- ricostruzione dell'albero ricevuto dal server.

```
Terminale - test del progetto

davide@map:~/Progetto-MAP-2026$ ./run-tests
Compilazione sorgenti e test JUnit 4.12 ...

JUnit version 4.12
.....
Time: 0.219

OK (84 tests)
```

Esecuzione JUnit: 84 test completati senza errori.

Note: i test GUI richiedono un ambiente grafico; database e rete vengono verificati anche tramite prove di integrazione.

Limiti

- la GUI visualizza il modello ma non guida una predizione;
- la modalità database richiede server e MySQL/MariaDB;
- host, porta del database e credenziali sono definiti nel codice del server;
- gli archivi Java richiedono classi compatibili tra salvataggio e caricamento.

Tecnologie

Java 8+, Swing/AWT, Socket TCP, Object Streams, JDBC, MySQL Connector/J 8.0.17 e JUnit 4.